

SHARIF  
UNIVERSITY OF TECHNOLOGY



---

# Signals & Systems

*Computer Assignment 2*

---

Aria Fateh Kerdari  
403110382

May 31, 2026

# 1 Properties of the Continuous-Time Fourier Transform

## 1.1 Intro to Fourier Transform

We load a sample sound signal to the workspace and then plot the FFT over frequencies.

```
1 load splat;  
2 y = y(1:8192);  
3 Y = fftshift(fft(y));
```

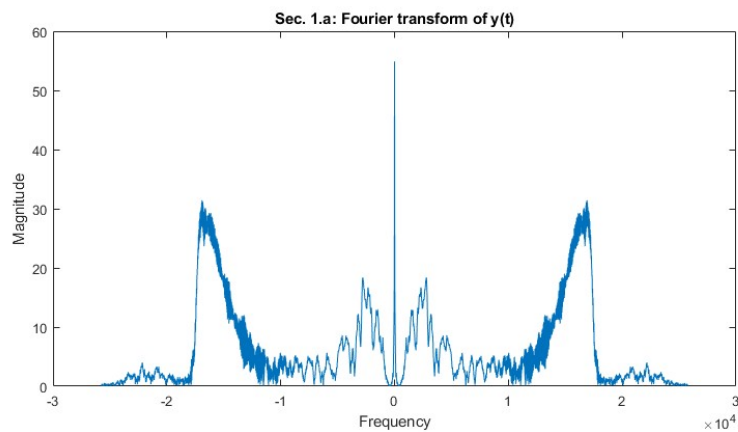


Figure 1: FFT of signal  $y(t)$ .

## 1.2 Conjugate of Fourier Transform

As we expected from theory the result sound is the time inverted signal.

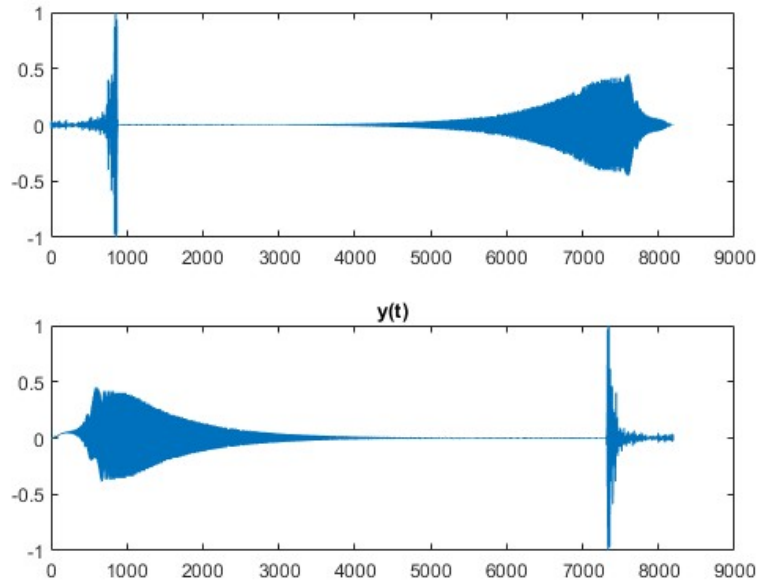


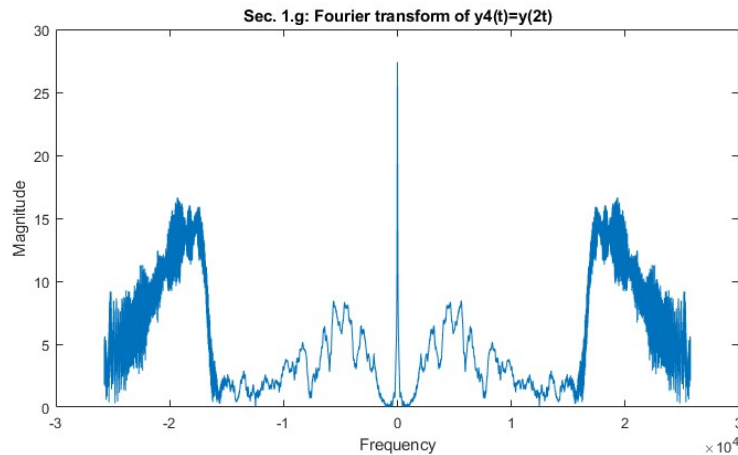
Figure 2: Conjugate in Fourier space is the time reversal transform of  $y(t)$ .

### 1.3 Magnitude or Phase?

After constructing `y2` and `y3` signals, we can say magnitude plays more crucial rule in the sound.

### 1.4 Constructing $y_4(t) = y(2t)$

The result signal is compressed by factor 2 in time space. But in frequency, signal is expanded. Compare the following figure with the section 1.1 figure.

Figure 3: FFT of  $y_4(t)$ 

## 1.5 Expansion in time

We insert zeros between samples.

```

1 x = zeros(2.*fs,1);
2 n = 1:1:fs;
3 x(2.*n) = y(n);

```

Then, with built-in function (`filter`)<sup>1</sup> we obtain the interpolated signal. The result of FFT shows us that expansion in time corresponds to compression in frequency. In `y5` lower frequencies have higher magnitude.

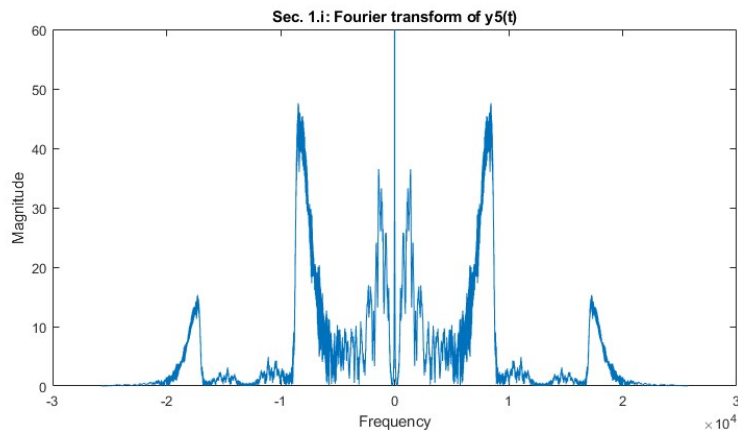
```

1 h = [1 2 1]./2;
2 y5 = filter(h, 1, x);
3 Y5 = fftshift(fft(y5));

```

<sup>1</sup>Filters are data processing techniques that can smooth out high-frequency fluctuations in data or remove periodic trends of a specific frequency from data. In MATLAB®, the `filter` function filters a vector of data `x` according to the following difference equation, which describes a tapped delay-line filter.

$$a(1)y(n) = \sum_{m=1}^{N_b} b(m)x(n-m+1) - \sum_{m=2}^{N_a} a(m)y(n-m+1)$$

Figure 4: FFT of  $y_5(t)$ 

## 2 Amplitude Modulation

In the figure (5), dash and dot signals are plotted. These signals are used to construct the Morse code. In the figure (6), **z** is drawn using dash and dot.

### 2.1 Filter Characteristic

The Bode plot of the filter is shown in figure (7). The line of code that gives us the Bode plot is as follows. **bf** and **af** are the transfer function coefficients.

```
1 ferqs(bf, af);
```

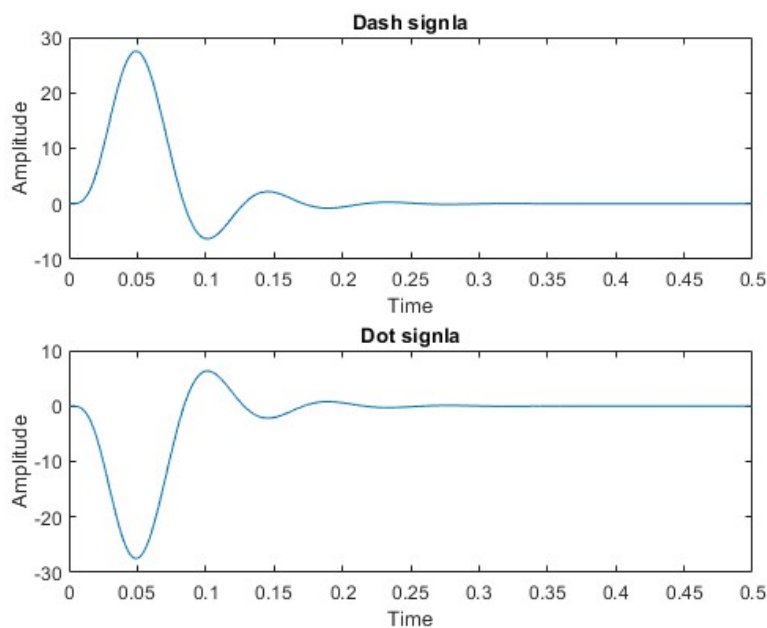


Figure 5: **dash** and **dot** signal which are used to construct the Morse code.

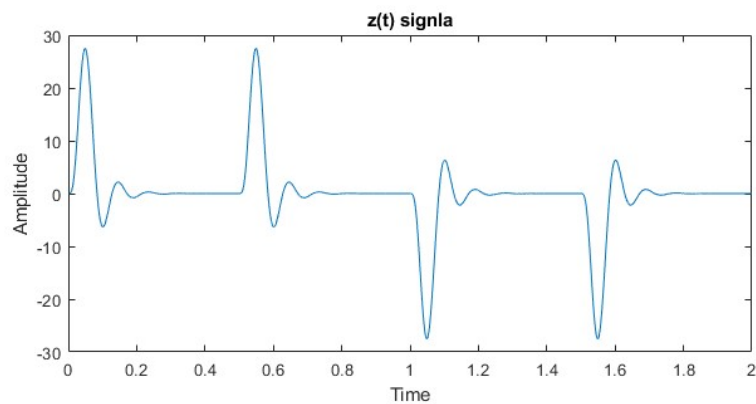


Figure 6: Corresponding **Z** in the Morse code.

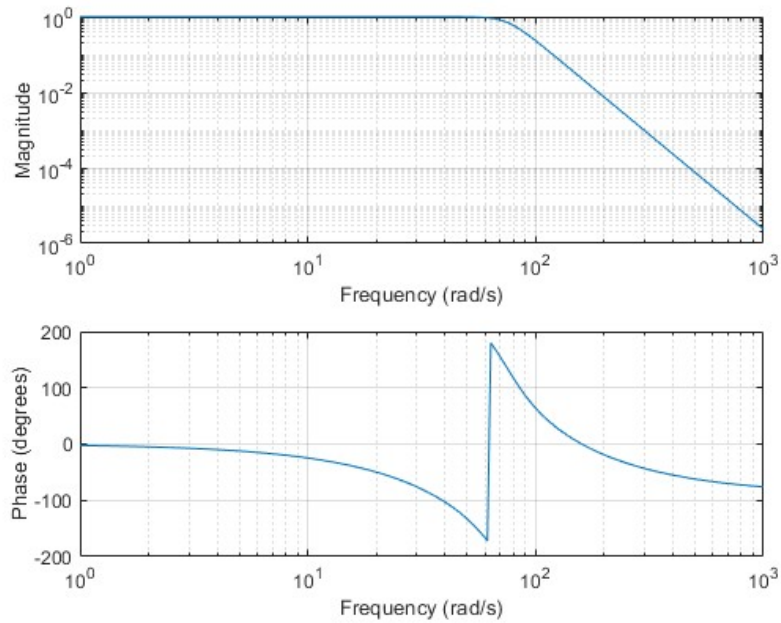


Figure 7: The filter Bode plot

## 2.2 Applying the Filter on `dash` and `dot`

After applying low-pass filter to dash and dot signals, the result is shown in figure (8).

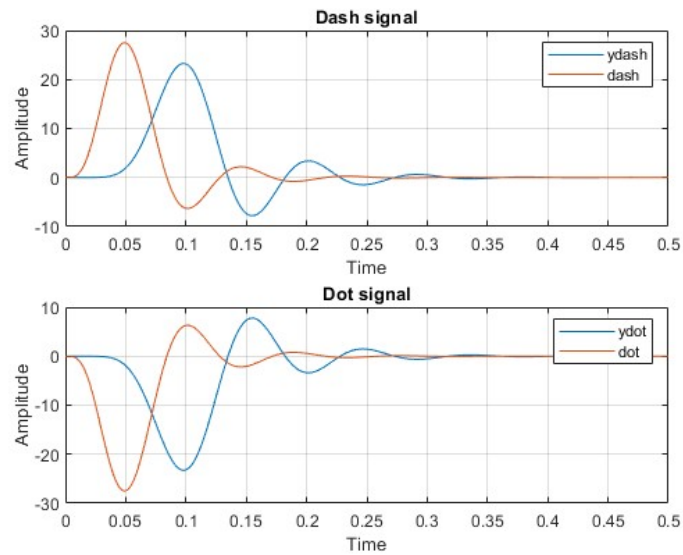


Figure 8: Dash and Dot signals

## 2.3 Modulation

If we define a new signal,  $y$  as following equation, most of the energy of resulting signal would be out of the pass-band.

```

1 y = dash.*cos(2*pi*f1.*t(1:length(dash)));
2 yo = lsim(bf,af,y,t(1:length(dash)));

```

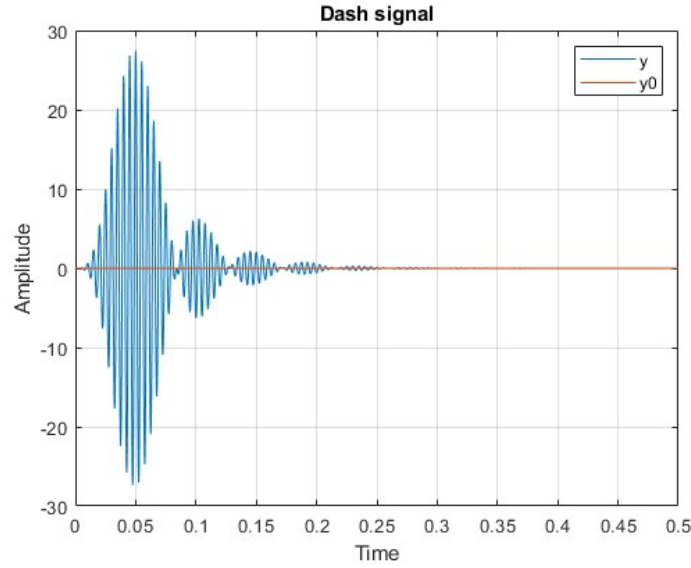


Figure 9: Signals y and yo.

## 2.4 Theory

- $y_1 = m(t) \cos(2\pi f_1 t)^2$  :

$$\cos(\omega_1 t)^2 = \frac{1}{2}(1 + \cos(2\omega_1 t))$$

$$\mathcal{F}\{y_1(t)\} = \frac{1}{2}M(j\omega) + \frac{1}{4}(M(j(\omega - 2\omega_1)) + M(j(\omega + 2\omega_1)))$$

- $y_2(t) = m(t) \cos(\omega_1) \sin(\omega_1 t)$  :

$$y_2(t) = \frac{1}{2}m(t) \sin(2\omega_1 t)$$

$$\mathcal{F}\{y_2(t)\} = \frac{1}{4j}(M(j(\omega - 2\omega_1)) - M(j(\omega + 2\omega_1)))$$

- $y_3(t) = m(t) \cos(\omega_1 t) \cos(\omega_2 t)$  :

$$y_3(t) = \frac{1}{2}m(t) \cos((\omega_1 - \omega_2)t) + \frac{1}{2}m(t) \cos((\omega_1 + \omega_2)t)$$

$$\mathcal{F}\{y_3(t)\} = \frac{1}{4} \left[ M(j(\omega - (\omega_1 - \omega_2))) + M(j(\omega + (\omega_1 - \omega_2))) \right. \\ \left. + M(j(\omega - (\omega_1 + \omega_2))) + M(j(\omega + (\omega_1 + \omega_2))) \right]$$

## 2.5 Extracting Messages

After multiplying  $\cos(\omega_1 t)$ , we apply the filter and as it is already calculated,

$$LPF\{x(t) \cos(\omega_1 t)\} = \frac{1}{2} m_1(t).$$

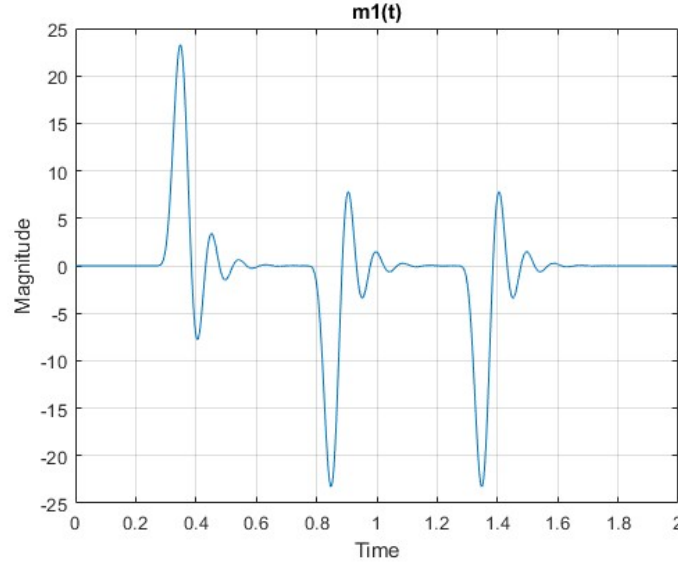


Figure 10: The first letter of the message (`m1`).

Then with similar methods, `m2` and `m3` are obtained.

```

1 m1 = 2.*lsim(bf,af,x.*cos(2*pi*f1.*t(1:length(x))),t(1:length(x)));
2 m2 = 2.*lsim(bf,af,x.*sin(2*pi*f2.*t(1:length(x))),t(1:length(xo)));
3 m3 = 2.*lsim(bf,af,x.*sin(2*pi*f1.*t(1:length(x))),t(1:length(xo)));

```

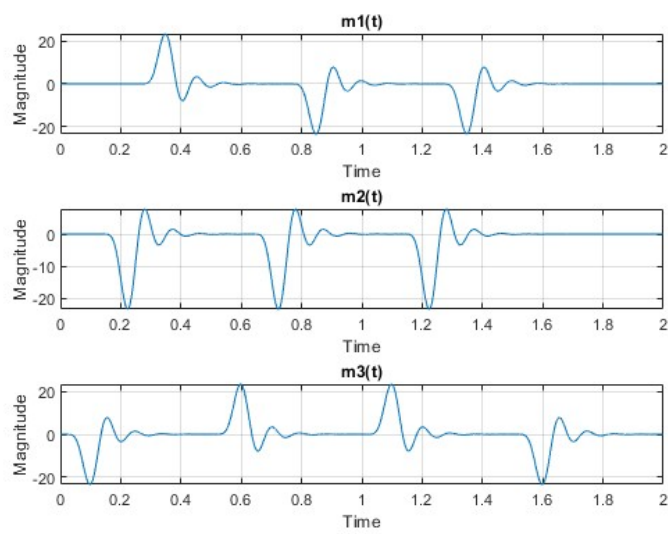


Figure 11: Demodulated message.